TECH DEBT

ELEVATED

ON-CALL ENGINEER QUITS

Your most experienced engineer — the only person who truly understands how the core system works — has handed in their notice this morning. They are leaving in two weeks. The institutional knowledge in their head is not written down anywhere.

The 'bus factor' (or 'truck factor') measures how many people need to leave before a system becomes unmaintainable. A bus factor of one is a critical organisational risk. The DevOps practices that reduce it include: runbooks and operational documentation stored in version control alongside the code; architecture decision records (ADRs) capturing why key decisions were made; pair programming and regular knowledge-sharing sessions; meaningful on-call rotation across the whole team (so operational knowledge is distributed); and code review practices that require reviewers to genuinely understand changes, not just approve them. Chronic on-call burnout — where one person handles most incidents because they are the only one who can — is both a cause and a symptom of high bus factor.

A bus factor of one is a ticking clock. Technical debt is not just in the code. It is in the people who understand the code.

TEAM DISCUSSION

Is your system documented well enough for someone new to operate it safely? What is your team's knowledge bus factor?